

Guía Completa de Docker para Principiantes

Índice

- 1. [Introducción](#)
- 2. [Conceptos Fundamentales](#)
- 3. [Primeros Pasos](#)
- 4. [Comandos Esenciales](#)
- 5. [Gestión de Contenedores](#)
- 6. [Gestión de Imágenes](#)
- 7. [Gestión de Volúmenes](#)
- 8. [Gestión de Redes](#)
- 9. [Docker Compose](#)
- 10. [Buenas Prácticas](#)
- 11. [Comandos Útiles de Emergencia](#)

Introducción

Docker es una plataforma de contenedorización que permite empaquetar aplicaciones con todas sus dependencias en contenedores ligeros y portables. Imagina que es como un barco container de carga: metes todo lo que tu aplicación necesita (código, librerías, configuraciones) en un "contenedor" que puede viajar a cualquier lugar y funcionar igual everywhere.

Esta guía está diseñada para principiantes absolutos. Aquí aprenderás:

- Qué es cada concepto de Docker
- Para qué sirve cada comando
- Cuándo usar cada opción

Conceptos Fundamentales (Explicado para Principiantes)

Antes de usar comandos, es importante entender qué es cada cosa:

Término	Explicación Simple
Imagen	Es como una "receta" o molde. Es una plantilla de solo lectura que contiene todo lo necesario para ejecutar una aplicación (código, librerías, sistema operativo mínimo). Piensa en ella como un plano para construir casas.
Contenedor	Es la "casa" construida a partir del plano. Es una instancia ejecutable de una imagen. Cuando ejecutas una imagen, creas un contenedor. Varios contenedores pueden usar la misma imagen.
Volumen	Es como un disco duro externo para el contenedor. Los datos que guardas aquí persisten incluso si eliminas el contenedor. Es útil para bases de datos y archivos que necesitas mantener.
Red	Es el "cable de red" que conecta contenedores entre sí o con el mundo exterior. Permite que dos contenedores hablen entre ellos o que accedas a tu aplicación desde internet.
Dockerfile	Es un archivo de texto donde escribes las instrucciones para crear tu propia imagen personalizada. Es como una receta de cocina escrita por ti.
Docker Compose	Es una herramienta que te permite definir y ejecutar aplicaciones con múltiples contenedores (por ejemplo: tu web + base de datos + caché) usando un solo archivo de configuración.

Primeros Pasos

Verificar que Docker está Instalado

Estos comandos te ayudan a confirmar que Docker está funcionando correctamente:

```
docker --version
```

Qué hace: Muestra la versión de Docker que tienes instalada. **Para qué sirve:** Confirmar que Docker se instaló correctamente. Deberías ver algo como "Docker version 24.0.0"

```
docker-compose --version
```

Qué hace: Muestra la versión de Docker Compose. **Para qué sirve:** Docker Compose viene separado en algunas instalaciones, verifica que lo tienes.

```
docker info
```

Qué hace: Muestra información detallada del sistema Docker. **Para qué sirve:** Ver cuántos contenedores tienes, imágenes, volúmenes, memoria usada, y configuración general de Docker. Es muy útil para diagnosticar problemas.

Habilitar Docker al Inicio del Sistema

```
sudo systemctl enable docker
```

Qué hace: Configura Docker para que se inicie automáticamente cuando enciendas el servidor. **Para qué sirve:** No tendrás que iniciar Docker manualmente cada vez que reinicies la máquina.

```
sudo systemctl start docker
```

Qué hace: Inicia el servicio de Docker ahora mismo. **Para qué sirve:** Si Docker no está corriendo, este comando lo inicia.

Ejecutar Docker sin sudo (RECOMENDADO)

Por defecto, Docker requiere permisos de administrador. Para no tener que escribir "sudo" antes de cada comando:

```
sudo usermod -aG docker $USER
```

Qué hace: Añade tu usuario actual al grupo "docker". **Para qué sirve:** Te permite ejecutar comandos docker sin ser root. **Importante:** Después de ejecutar esto, CIERRA SESIÓN y vuelve a entrar (o reinicia) para que los cambios tengan efecto.

Comandos Esenciales

Información del Sistema

```
docker version
```

Qué hace: Muestra la versión del cliente y del servidor Docker. **Para qué sirve:** Verificar que tanto el cliente (lo que tú ejecutas) como el servidor Docker (el motor que corre contenedores) están instalados y funcionando.

```
docker info
```

Qué hace: Muestra información completa del sistema Docker. **Para qué sirve:** Ver cuántos contenedores hay, imágenes descargadas, espacio en disco usado, drivers de almacenamiento y red configurados, etc.

```
docker system df
```

Qué hace: Muestra cuánto espacio en disco están usando los recursos de Docker. **Para qué sirve:** Ver cuanto espacio ocupan las imágenes, contenedores y volúmenes. Si el disco se llena, esto te ayuda a saber qué limpieza hacer.

```
docker system prune
```

Qué hace: Limpia recursos no usados (contenedores detenidos, redes sin usar, imágenes colgadas). **Para qué sirve:** Recuperar espacio en disco. OJO: Solo elimina lo que no se está usando activamente.

Gestión de Contenedores

Los contenedores son el corazón de Docker. Son las "cajas" donde corren tus aplicaciones.

Ver Contenedores

```
docker ps
```

Qué hace: Lista solo los contenedores que están en ejecución actualmente. **Para qué sirve:** Ver qué aplicaciones están corriendo en este momento. Muestra el ID corto, nombre, imagen, estado, puertos.

```
docker ps -a
```

Qué hace: Lista TODOS los contenedores, incluyendo los que están detenidos. **Para qué sirve:** Ver el historial de contenedores. Los contenedores detenidos ocupan espacio y pueden acumularse.

```
docker ps -l
```

Qué hace: Muestra solo el ÚLTIMO contenedor que creaste (incluyendo si está detenido). **Para qué sirve:** Encontrar rápidamente el contenedor más reciente para operarlo.

```
docker ps --format "table {{.ID}}\t{{.Names}}\t{{.Status}}"
```

Qué hace: Lista contenedores con formato personalizado, solo mostrando ID, Nombre y Estado. **Para qué sirve:** Hacer la salida más legible. Puedes cambiar los campos que quieres ver.

Crear y Ejecutar Contenedores

El comando principal para crear y ejecutar contenedores es `docker run`. Aquí te explico cada opción:

```
docker run <nombre_imagen>
```

Qué hace: Descarga la imagen (si no la tienes) y crea un nuevo contenedor a partir de ella. **Para qué sirve:** Lanzar una aplicación en un contenedor. Ejemplo: `docker run nginx` lanza un servidor web nginx. **Nota:** Si la imagen no existe localmente, Docker la descarga automáticamente desde Docker Hub.

```
docker run --name mi_contenedor <nombre_imagen>
```

Qué hace: Asigna un nombre personalizado al contenedor en lugar de generar uno aleatorio. **Para qué sirve:** Es más fácil recordar y trabajar con "mi_contenedor" que con "angry_knuth_123". **Ejemplo:** `docker run --name mi_web nginx`

```
docker run -it <nombre_imagen> /bin/bash
```

Qué hace:

- `-i` (interactive): Mantiene la entrada estándar abierta
- `-t` (tty): Asigna una terminal
- `/bin/bash`: Ejecuta una terminal bash dentro del contenedor **Para qué sirve:** Abrir una terminal interactiva dentro del contenedor. Útil para explorar el sistema de archivos o depurar. **Ejemplo:** `docker run -it ubuntu /bin/bash` te da una terminal de Ubuntu dentro del contenedor.

```
docker run -d <nombre_imagen>
```

Qué hace:

- `-d` (detach): Ejecuta el contenedor en segundo plano (como servicio) **Para qué sirve:** Lanzar el contenedor y seguir usando la terminal. El contenedor corre en background. **Ejemplo:** `docker run -d nginx` lanza nginx pero te devuelve el control de la terminal.

```
docker run -p 8080:80 <nombre_imagen>
```

Qué hace:

- `-p` (publish): Mapea puertos del host al contenedor
- Formato: `puerto_host:puerto_contenedor` **Para qué sirve:** Acceder a la aplicación del contenedor desde fuera. El contenedor nginx usa el puerto 80 internamente, pero lo exponemos al puerto 8080 de tu servidor. **Ejemplo:** `docker run -p 8080:80 nginx` - Accede a nginx en http://tu_servidor:8080

```
docker run -e MI_VARIABLE=valor <nombre_imagen>
```

Qué hace:

- `-e` (env): Establece variables de entorno dentro del contenedor **Para qué sirve:** Pasar configuración a la aplicación. Muchas imágenes buscan variables específicas. **Ejemplo:** `docker run -e MYSQL_ROOT_PASSWORD=secreto mysql`

```
docker run -v /ruta/host:/ruta/contenedor <nombre_imagen>
```

Qué hace:

- `-v` (volume): Monta un volumen (directorio o archivo) dentro del contenedor
- Formato: `ruta_host:ruta_contenedor` **Para qué sirve:** Persistir datos fuera del contenedor o compartir archivos entre host y contenedor. **Ejemplo:**

```
docker run -v /mi/datos:/app/data mi_app - Los datos en /app/data del contenedor están también en /mi/datos del servidor.
```

```
docker run --restart=always <nombre_imagen>
```

Qué hace: Configura el contenedor para que se reinicie automáticamente si se detiene o si el servidor se reinicia. **Para qué sirve:** Servicios que deben estar siempre disponibles. Ideal para producción. **Opciones:** `--restart=no` (por defecto), `--restart=on-failure`, `--restart=unless-stopped`

```
docker run -d --name mi_servidor -p 8080:80 -e ENV=production -v /data:/app nginx
```

Qué hace: Ejemplo completo combinando múltiples opciones. **Para qué sirve:** Este comando:

- `-d` : Ejecuta en background
- `--name` : Lo llama "mi_servidor"
- `-p 8080:80` : Expone el puerto 80 al 8080
- `-e ENV=production` : Variable de entorno
- `-v /data:/app` : Monta volumen

Iniciar/Detener Contenedores

```
docker start <nombre_o_id>
```

Qué hace: Inicia un contenedor que estaba detenido. **Para qué sirve:** Reanudar un contenedor que paraste o que felló. El contenedor mantiene su configuración. **Ejemplo:** `docker start mi_web`

```
docker stop <nombre_o_id>
```

Qué hace: Detiene gracefully un contenedor en ejecución (envía señal SIGTERM, luego SIGKILL). **Para qué sirve:** Parar una aplicación de forma controlada, permitiendo que termine procesos correctamente. **Ejemplo:** `docker stop mi_web`

```
docker restart <nombre_o_id>
```

Qué hace: Detiene y luego inicia el contenedor. **Para qué sirve:** Reiniciar una aplicación (útil después de cambios de configuración).

```
docker pause <nombre_o_id>
```

Qué hace: Pausa todos los procesos dentro del contenedor. **Para qué sirve:** Suspender temporalmente un contenedor sin perder su estado. Los procesos quedan "congelados".

```
docker unpause <nombre_o_id>
```

Qué hace: Reanuda un contenedor que estaba en pausa. **Para qué sirve:** Continuar la ejecución después de una pausa.

Eliminar Contenedores

```
docker rm <nombre_o_id>
```

Qué hace: Elimina un contenedor DELGADO (que no está corriendo). **Para qué sirve:** Liberar espacio y limpiar contenedores que ya no necesitas. **Nota:** Si el contenedor está corriendo, te dará error.

```
docker rm -f <nombre_o_id>
```

Qué hace: Fuerza la eliminación aunque el contenedor esté corriendo. **Para qué sirve:** Eliminar contenedores que no responden o que necesitas quitar urgentemente. **Advertencia:** Puede causar pérdida de datos si había procesos sin guardar.

```
docker rm $(docker ps -aq)
```

Qué hace:

- `docker ps -aq` : Lista solo los IDs de todos los contenedores
- `$( )` : Toma esa lista y la pasa a `docker rm` **Para qué sirve:** Eliminar TODOS los contenedores detenidos de una vez.

```
docker container prune
```

Qué hace: Elimina todos los contenedores detenidos. **Para qué sirve:** Limpieza rápida. Te pregunta confirmación (usa `-f` para forzar).

Ejecutar Comandos en un Contenedor

A veces necesitas ejecutar comandos dentro de un contenedor que ya está corriendo:

```
docker exec -it <contenedor> bash
```

Qué hace: Abre una terminal interactiva dentro de un contenedor en ejecución. **Para qué sirve:** Depurar, instalar cosas, ver archivos, configurar servicios dentro del contenedor. **Ejemplo:** `docker exec -it mi_web bash` - Entra al contenedor "mi_web"

```
docker exec -it <contenedor> ls
```

Qué hace: Ejecuta un comando específico (`ls`) dentro del contenedor y muestra el resultado. **Para qué sirve:** Ejecutar un comando sin entrar a la terminal. Útil para scripts.

```
docker exec -d <contenedor> comando
```

Qué hace: Ejecuta un comando en modo detach (background). **Para qué sirve:** Ejecutar procesos en segundo plano dentro del contenedor.

Ver Logs de un Contenedor

Los logs son fundamentales para diagnosticar problemas:

```
docker logs <contenedor>
```

Qué hace: Muestra los logs (salida estándar) del contenedor. **Para qué sirve:** Ver qué está haciendo la aplicación, errores, mensajes de inicio. **Ejemplo:** `docker logs mi_nginx`

```
docker logs -f <contenedor>
```

Qué hace: Muestra logs y se queda "pegado" mostrando nuevos mensajes en tiempo real. **Para qué sirve:** Monitorear actividad en vivo. Como `tail -f`. **Salir:** Presiona Ctrl+C

```
docker logs --tail 100 <contenedor>
```

Qué hace: Muestra solo las últimas 100 líneas de logs. **Para qué sirve:** Ver lo más reciente sin abrumarte con todo el historial.

```
docker logs --since 1h <contenedor>
```

Qué hace: Muestra logs desde la última hora. **Para qué sirve:** Investigar problemas recientes.

Inspeccionar Contenedores

```
docker inspect <contenedor>
```

Qué hace: Muestra información detallada en formato JSON sobre el contenedor. **Para qué sirve:** Ver configuración completa: IP, mounts, redes, variables de entorno, estado, etc. **Nota:** La salida es extensa, usa `--format` para filtrar.

```
docker inspect --format='{{.NetworkSettings.IPAddress}}' <contenedor>
```

Qué hace: Extrae solo la dirección IP del contenedor. **Para qué sirve:** Obtener rápidamente información específica sin parsear JSON.

```
docker stats <contenedor>
```

Qué hace: Muestra uso de recursos en tiempo real (CPU, memoria, red, disco). **Para qué sirve:** Monitorear rendimiento, detectar problemas de memoria o CPU. **Salir:** Ctrl+C

```
docker top <contenedor>
```

Qué hace: Lista los procesos que están corriendo dentro del contenedor. **Para qué sirve:** Ver qué procesos exactos están ejecutándose (como `ps` dentro del contenedor).

```
docker diff <contenedor>
```

Qué hace: Muestra los cambios en el sistema de archivos desde que se creó el contenedor. **Para qué sirve:** Ver qué archivos fueron modificados, añadidos o eliminados.

Redes y Puertos

```
docker port <contenedor>
```

Qué hace: Muestra los puertos que el contenedor tiene expuestos y cómo están mapeados. **Para qué sirve:** Ver rápidamente qué puertos están accesibles.

```
docker network ls
```

Qué hace: Lista todas las redes de Docker. **Para qué sirve:** Ver qué redes existen (bridge, host, overlay, etc.).

```
docker network inspect <red>
```

Qué hace: Muestra detalles de una red específica (contenedores conectados, configuración). **Para qué sirve:** Diagnosticar problemas de red entre contenedores.

Gestión de Imágenes

Las imágenes son las "plantillas" usadas para crear contenedores.

Listar Imágenes

```
docker images
```

Qué hace: Lista todas las imágenes que tienes descargadas localmente. **Para qué sirve:** Ver qué imágenes tienes disponibles para crear contenedores. **Columnas:** Repository, Tag, Image ID, Created, Size

```
docker images -a
```

Qué hace: Incluye las imágenes intermediarias (capas intermedias que Docker crea al construir). **Para qué sirve:** Ver todo el historial de imágenes.

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

Qué hace: Formato de salida personalizado. **Para qué sirve:** Make it más legible, mostrando solo lo que te interesa.

Buscar Imágenes

```
docker search <término>
```

Qué hace: Busca imágenes en Docker Hub (el repositorio público de Docker). **Para qué sirve:** Encontrar imágenes oficiales o de la comunidad. **Ejemplo:** `docker search nginx` - Busca imágenes relacionadas con nginx.

```
docker search --limit 10 <término>
```

Qué hace: Limita los resultados a 10. **Para qué sirve:** No saturarte con resultados si solo necesitas los más populares.

Descargar Imágenes

```
docker pull <nombre_imagen>
```

Qué hace: Descarga una imagen desde Docker Hub (o otro registry). **Para qué sirve:** Obtener una imagen sin crear un contenedor. Útil para preparar el entorno. **Ejemplo:** `docker pull ubuntu` - Descarga la última versión de Ubuntu.

```
docker pull <nombre_imagen>:etiqueta
```

Qué hace: Descarga una versión específica de la imagen. **Para qué sirve:** Usar versiones específicas para consistencia y reproducibilidad. **Ejemplo:** `docker pull nginx:1.25` - Descarga nginx versión 1.25

```
docker pull <nombre_imagen>:latest
```

Qué hace: Descarga explícitamente la última versión. **Para qué sirve:** Asegurarte de tener la versión más reciente.

Eliminar Imágenes

```
docker rmi <nombre_imagen>
```

Qué hace: Elimina una imagen local. **Para qué sirve:** Liberar espacio en disco. La imagen no puede estar siendo usada por algún contenedor. **Nota:** Si tienes contenedores usando esa imagen, primero elimínalos.

```
docker rmi -f <nombre_imagen>
```

Qué hace: Fuerza la eliminación de la imagen aunque haya contenedores usándola. **Advertencia:** Puede dejar contenedores huérfanos o rotos.

```
docker rmi $(docker images -q)
```

Qué hace: Elimina TODAS las imágenes locales. **Para qué sirve:** Limpieza total. OJO: Necesitas recrear las imágenes después.

```
docker image prune
```

Qué hace: Elimina imágenes que no están asociadas a ningún contenedor. **Para qué sirve:** Limpieza automática de imágenes no usadas.

```
docker image prune -a
```

Qué hace: Elimina TODAS las imágenes no usadas. **Para qué sirve:** Recuperar máximo espacio.

Construir Imágenes

```
docker build -t mi_imagen:tag .
```

Qué hace:

- **build** : Construye una imagen
- **-t** : Etiqueta la imagen con nombre:tag
- **.** : Busca el Dockerfile en el directorio actual **Para qué sirve:** Crear tu propia imagen desde un Dockerfile. **Ejemplo:** `docker build -t mi_app:1.0 .`

```
docker build -f MiDockerfile -t mi_imagen:tag .
```

Qué hace: Usa un archivo Dockerfile con nombre diferente al estándar. **Para qué sirve:** Si tienes múltiples Dockerfiles o está en otro lugar.

```
docker build --build-arg VERSION=1.0 -t mi_imagen .
```

Qué hace: Pasa argumentos de construcción (ARG) al Dockerfile. **Para qué sirve:** Personalizar la construcción con variables.

Etiquetar Imágenes

```
docker tag imagen_original:tag nuevo_nombre:tag
```

Qué hace: Crea una nueva etiqueta para una imagen existente (no crea una copia). **Para qué sirve:** Renombrar o preparar una imagen para subirla a un registry. **Ejemplo:** `docker tag mi_app:latest mi_usuario/mi_app:1.0`

Subir Imágenes

```
docker push <nombre_imagen>:tag
```

Qué hace: Sube una imagen a un registry (Docker Hub por defecto). **Para qué sirve:** Compartir tus imágenes o guardarlas en un registry privado. **Ejemplo:** `docker push mi_usuario/mi_app:1.0`

Historial de una Imagen

```
docker history <nombre_imagen>
```

Qué hace: Muestra todas las capas que forman la imagen. **Para qué sirve:** Entender cómo se construyó la imagen, identificar capas innecesarias para optimizar.

Gestión de Volúmenes

Los volúmenes sirven para persistir datos fuera de los contenedores.

Listar Volúmenes

```
docker volume ls
```

Qué hace: Lista todos los volúmenes creados. **Para qué sirve:** Ver qué volúmenes existen y cuánto espacio ocupan.

```
docker volume ls -f dangling=true
```

Qué hace: Muestra solo volúmenes no usados por ningún contenedor. **Para qué sirve:** Identificar volúmenes huérfanos que puedes eliminar.

Crear Volúmenes

```
docker volume create mi_volumen
```

Qué hace: Crea un volumen nombrado vacío. **Para qué sirve:** Preparar un volumen antes de usarlo, o para usar con docker-compose.

Eliminar Volúmenes

```
docker volume rm mi_volumen
```

Qué hace: Elimina un volumen específico. **Para qué sirve:** Liberar espacio. ADVERTENCIA: Se pierden todos los datos.

```
docker volume prune
```

Qué hace: Elimina todos los volúmenes no usados. **Para qué sirve:** Limpieza masiva. Pide confirmación.

Inspeccionar Volúmenes

```
docker volume inspect mi_volumen
```

Qué hace: Muestra detalles del volumen (donde se almacena, driver, etc.). **Para qué sirve:** Ver la ubicación real de los datos en el host.

Tipos de Volúmenes

Volumen Nombrado (recomendado para producción):

```
docker run -v mi_volumen:/ruta/datos imagen
```

Qué hace: Monta un volumen nombrado dentro del contenedor. **Para qué sirve:** Persistencia de datos gestionada por Docker.

Bind Mount (para desarrollo):

```
docker run -v /ruta/host:/ruta/contenedor imagen
```

Qué hace: Monta un directorio del host directamente en el contenedor. **Para qué sirve:** Compartir archivos entre host y contenedor (útil para desarrollo).

Solo Lectura:

```
docker run -v /data:/data:ro imagen
```

Qué hace: Monta el volumen en modo solo lectura. **Para qué sirve:** Evitar que el contenedor modifique datos importantes.

tmpfs (en memoria):

```
docker run --tmpfs /ruta imagen
```

Qué hace: Guarda datos en memoria RAM en lugar del disco. **Para qué sirve:** Datos sensibles o temporales que no deben persistir.

Gestión de Redes

Las redes permiten que los contenedores se comuniquen entre sí y con el exterior.

Listar Redes

```
docker network ls
```

Qué hace: Lista todas las redes de Docker. **Para qué sirve:** Ver redes existentes (bridge, host, overlay, none).

Crear Red

```
docker network create mi_red
```

Qué hace: Crea una red bridge por defecto. **Para qué sirve:** Aislar contenedores en una red privada.

```
docker network create --driver bridge mi_red_bridge
```

Qué hace: Crea una red bridge explícitamente. **Para qué sirve:** Red para contenedores que necesitan comunicarse.

```
docker network create --driver overlay mi_red_overlay
```

Qué hace: Crea una red overlay (para Docker Swarm). **Para qué sirve:** Comunicación entre contenedores en múltiples máquinas.

Conectar/Desconectar Contenedores

```
docker network connect mi_red contenedor
```

Qué hace: Conecta un contenedor existente a una red. **Para qué sirve:** Añadir comunicación a un contenedor después de creado.

```
docker network disconnect mi_red contenedor
```

Qué hace: Desconecta un contenedor de una red. **Para qué sirve:** Aislar un contenedor.

Eliminar Red

```
docker network rm mi_red
```

Qué hace: Elimina una red. **Para qué sirve:** Limpieza. No puede tener contenedores conectados.

```
docker network prune
```

Qué hace: Elimina todas las redes no usadas. **Para qué sirve:** Limpieza automática.

Docker Compose

Docker Compose te permite definir aplicaciones multicontenedor en un solo archivo.

Comandos Básicos

```
docker-compose up
```

Qué hace: Crea y ejecuta todos los servicios definidos en docker-compose.yml. **Para qué sirve:** Lanzar toda tu aplicación con un solo comando. **Nota:** Si las imágenes no existen, las construye.

```
docker-compose up -d
```

Qué hace: Ejecuta en modo detach (background). **Para qué sirve:** Ejecutar en segundo plano para mantener la terminal libre.

```
docker-compose down
```

Qué hace: Detiene y elimina todos los contenedores, redes y volúmenes definidos. **Para qué sirve:** Limpiar el entorno cuando terminas de desarrollar.

```
docker-compose down -v
```

Qué hace: Incluye eliminación de volúmenes. **Para qué sirve:** Limpieza total (cuidado: borra datos).

```
docker-compose down --rm all
```

Qué hace: También elimina las imágenes creadas. **Para qué sirve:** Limpieza completa.

Ver Estado

```
docker-compose ps
```

Qué hace: Muestra el estado de todos los servicios. **Para qué sirve:** Ver qué está corriendo.

```
docker-compose logs
```

Qué hace: Muestra logs de todos los servicios. **Para qué sirve:** Ver qué está pasando.

```
docker-compose logs -f
```

Qué hace: Logs en tiempo real. **Para qué sirve:** Monitorear actividad.

```
docker-compose logs servicio
```

Qué hace: Logs de un servicio específico. **Para qué sirve:** Enfocarte en un componente.

Otros Comandos

```
docker-compose build
```

Qué hace: Construye las imágenes definidas con "build" en el compose. **Para qué sirve:** Preparar imágenes antes de ejecutar.

```
docker-compose pull
```

Qué hace: Descarga las imágenes definidas. **Para qué sirve:** Actualizar imágenes.

```
docker-compose restart
```

Qué hace: Reinicia todos los servicios. **Para qué sirve:** Reiniciar tras cambios de configuración.

```
docker-compose stop
```

Qué hace: Detiene los servicios sin eliminarlos. **Para qué sirve:** Pausar temporalmente.

```
docker-compose start
```

Qué hace: Inicia servicios detenidos. **Para qué sirve:** Reanudar tras stop.

Escalar Servicios

```
docker-compose up -d --scale servicio=3
```

Qué hace: Escala un servicio a 3 instancias. **Para qué sirve:** Balanceo de carga simple (nota: requiere configuración de load balancer).

Debug

```
docker-compose exec servicio bash
```

Qué hace: Abre terminal en un servicio específico. **Para qué sirve:** Depurar dentro de un servicio.

```
docker-compose config
```

Qué hace: Valida y muestra la configuración procesada. **Para qué sirve:** Verificar que el archivo está correcto.

Ejemplo docker-compose.yml Completo

```
# Versión del formato de compose
version: '3.8'

# Definición de servicios (contenedores)
services:
  # Servicio 1: Servidor Web
  web:
    # Imagen a usar (o build para construir)
    image: nginx:latest
    # Puertos a exponer (host:contenedor)
    ports:
      - "80:80"
    # Volúmenes a montar
    volumes:
      - ./html:/usr/share/nginx/html:ro
    # Dependencias (inicia primero api)
    depends_on:
      - api
    # Red a usar
    networks:
      - mi_red
    # Política de reinicio
    restart: unless-stopped

  # Servicio 2: API
  api:
    # Construir desde Dockerfile en ./api
    build: ./api
    # Variables de entorno
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgres://db:5432/app
    # Volumen nommée
    volumes:
      - datos:/app/data
    # Dependencias
    depends_on:
      - db
    # Red a usar
    networks:
      - mi_red
    restart: unless-stopped

  # Servicio 3: Base de datos
  db:
    image: postgres:15
    environment:
      - POSTGRES_DB=app
      - POSTGRES_USER=usuario
      - POSTGRES_PASSWORD=contraseña
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - mi_red
    restart: unless-stopped

# Definición de volúmenes nombrados
volumes:
  datos:
  postgres_data:

# Definición de redes
networks:
```

```
mi_red:
  driver: bridge
```

Buenas Prácticas

Nomenclatura

```
# ☑ Bien: nombre descriptivo
docker run --name mi_servidor_web_nginx
```

```
# ☒ Mal: nombre aleatorio
docker run --name xenial_duck
```

```
# Usar etiquetas para versiones
docker tag mi_imagen:latest mi_imagen:1.0.0
```

Limpieza Regular

```
# Limpiar todo lo no usado
docker system prune -a
```

```
# Incluyendo volúmenes
docker system prune --volumes
```

```
# Solo contenedores
docker container prune
```

```
# Solo imágenes
docker image prune
```

```
# Solo volúmenes
docker volume prune
```

```
# Solo redes
docker network prune
```

Seguridad

```
# No ejecutar como root
docker run -u 1000:1000 mi_imagen
```

```
# Limitar memoria
docker run --memory=512m mi_imagen
```

```
# Limitar CPUs
docker run --cpus=0.5 mi_imagen
```

```
# Solo lectura
docker run --read-only mi_imagen
```

```
# No almacenar secretos en imágenes
```

Optimización

Crear un `.dockerignore`:

```
node_modules
.git
*.log
.env
__pycache__
```

Comandos Útiles de Emergencia

Recuperar Acceso a un Contenedor

```
# Si perdiste acceso
docker exec -it <contenedor> /bin/sh

# Crear imagen de rescate
docker commit <contenedor> imagen_rescate
```

Recuperar Datos de un Contenedor Eliminado

```
# Ver volúmenes existentes
docker volume ls

# Montar volumen en un contenedor temporal
docker run --rm -v <volumen>:/data ubuntu cat /data/archivo
```

Reiniciar Docker

```
sudo systemctl restart docker
```

Verificar Estado

```
sudo systemctl status docker
journalctl -u docker -n 50
```

Encontrar Contenedor por Puerto

```
docker ps --filter "publish=80"
```

Copiar Archivos

```
# Del contenedor al host
docker cp mi_contenedor:/ruta/origen ./destino

# Del host al contenedor
docker cp ./origen mi_contenedor:/ruta/destino
```

Comandos de Referencia Rápida

Acción	Comando
Ver contenedores activos	<code>docker ps</code>
Ver todos los contenedores	<code>docker ps -a</code>

Iniciar contenedor Acción	docker start nombre Comando
Detener contenedor	docker stop nombre
Eliminar contenedor	docker rm nombre
Ver logs en vivo	docker logs -f nombre
Entrar al contenedor	docker exec -it nombre bash
Ver imágenes	docker images
Descargar imagen	docker pull nombre
Eliminar imagen	docker rmi nombre
Ver volúmenes	docker volume ls
Ver redes	docker network ls
Iniciar compose	docker-compose up -d
Detener compose	docker-compose down

Próximos Pasos Recomendados

- 1. **Practica:** Crea contenedores simples como nginx, redis, postgres
 - 2. **Aprende Dockerfile:** Escribe tus propias imágenes
 - 3. **Docker Compose:** Define aplicaciones multicontenedor
 - 4. **Docker Swarm:** Aprende orquestación básica
 - 5. **Registros privados:** Configura tu propio registry
-